Atelier5 : Formulaires et vues génériques de Django

I. Utilisation des vues générique Django

1. Afficher la liste des produits

1) Dans le fichier views.py de l'application « products », modifiez le code de la vue ProductListView qui doit hériter maintenant de la classe générique ListView de Django.
   - Remplacez l'implémentation actuelle de la vue par le code ci-dessous :

views.py 1 ×

backend > products > views.py > ...

```python
from django.shortcuts import render
from django.views import View
from .models import Product
from django.views.generic import ListView


class ProductsListView(ListView):
    model=Product
    template_name='products/product_list.html'
    context_object_name='produits'

```

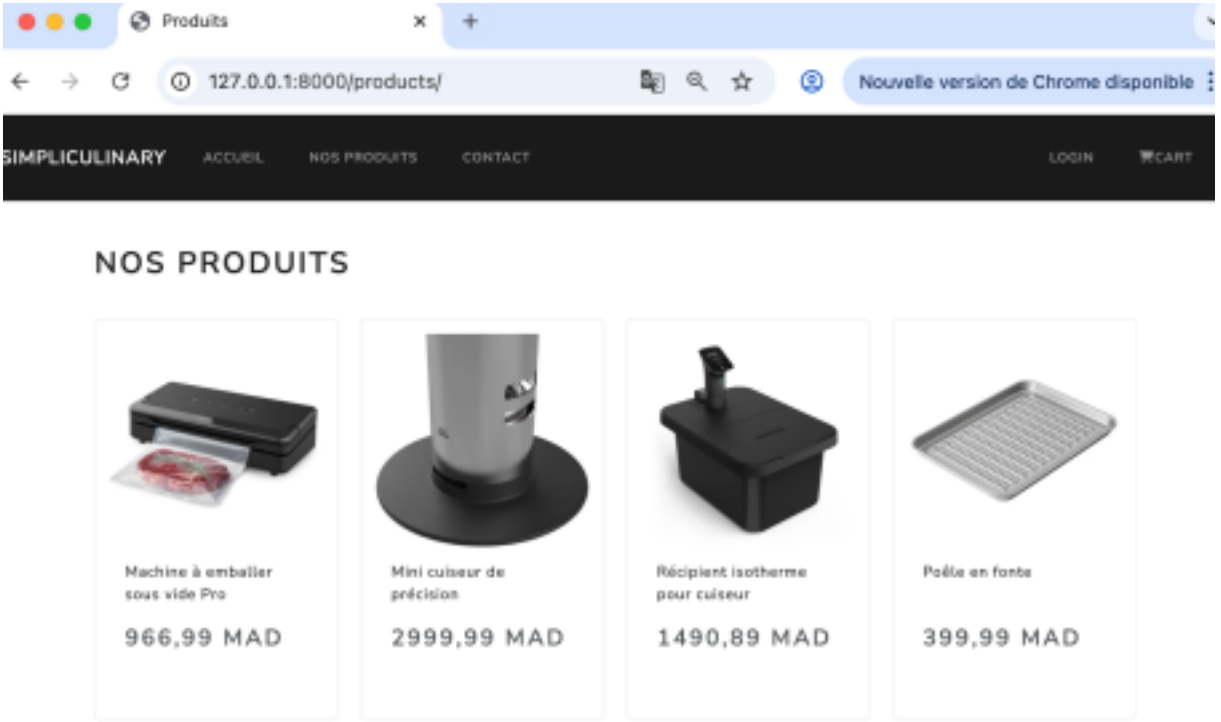
2) Modifier la template « products/product_list.html », pour afficher les produits récupérés depuis la base de données sous forme de grille de carte

product_list.html ×

backend > products > templates > products > product_list.html > ...

```html
{% extends 'base/base.html' %}
{% block title %}Produits{% endblock  %}
{% block content %}
    <div class="container my-5">
        <h1>Nos Produits</h1>
        <div class="row">
            {% for product in produits %}
                <div class="col-sm-12 col-md-6 col-lg-4 col-xl-3 py-3">
                    <div class="card bg-light my-3  p-3 rounded" style="height: 100%;">
                        <a href="{% url 'product_detail' product.id %}">
                            <img src="{{product.image.url}}" class="card-img" alt="{{product.name}}">
                        </a>
                        <div class="card-body">
                            <div class="card-title" style="height: 60px;">
                                <a href="{% url 'product_detail' product.id %}" class="text-decoration-none">
                                    <strong>{{product.name}}</strong>
                                </a>
                            </div>
                            <div class="card-text">
                                <h3>{{product.price}} MAD</h3>
                            </div>
                        </div>
                    </div>
                </div>
            {% endfor %}
        </div>
    </div>
{% endblock %}
```

3) Désormais, en cliquant sur le menu "Nos produits", vous verrez l'aperçu suivant :



2. Afficher les détails sur un produit spécifique

1) Dans le fichier views.py de l'application « products», créez une nouvelle vue appelée ProductDetailsView qui hérite de la classe générique DetailView de Django

views.py

backend > products > views.py > ProductDetailsView

```python
from django.shortcuts import render
from django.views import View
from .models import Product
from django.views.generic import DetailView,ListView


class ProductsListView(ListView):
    model=Product
    template_name='products/product_list.html'
    context_object_name='produits'

class ProductDetailsView(DetailView):
    template_name="products/product_detail.html"
    model = Product
    context_object_name='product'

```

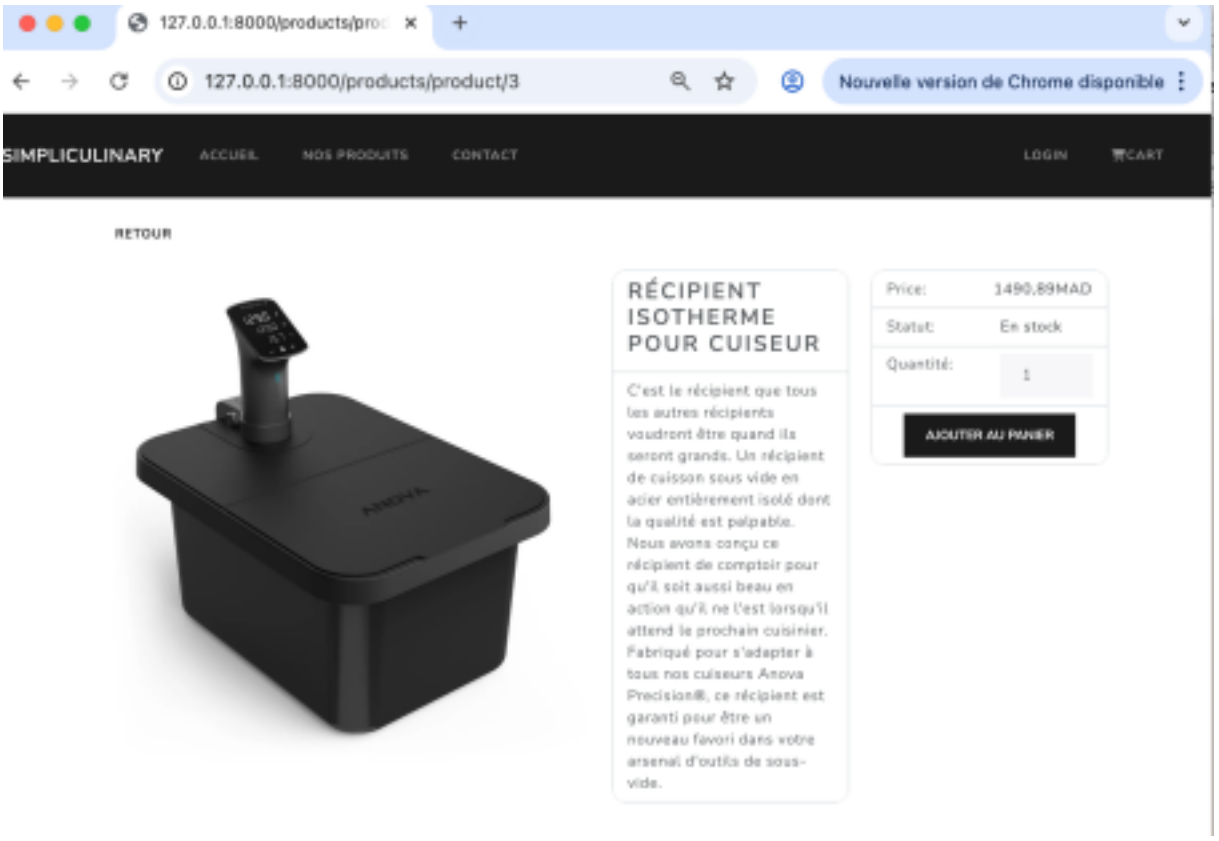
2) Associé la view ProductDetail **à l'url suivant (ligne 5)** :

urls.py

backend > products > urls.py > ...

```
1  from django.urls import path
2  from .views import ProductDetailsView,ProductsListView
3  urlpatterns = [
4      path('',ProductsListView.as_view(),name='products_list'),
5      path('product/<int:pk>',ProductDetailsView.as_view(),name='product_detail'),
6  ]
```

3) Créez une nouvelle template « productDetail.html», au sein du dossiers templates de l'application «products » pour afficher les détails du produit comme l'aperçu suivant : − Le code à définir dans la template est dans **la page 4**



product_detail.html

backend > products > templates > products > product_detail.html > ...

```
1  {% extends 'base/base.html' %}
2
3  {% block content %}
```

II. Gestion du panier :

1. Ajouter un produit au panier

1) Dans l'application cart créez une nouvelle vue appelée **AddToCartView,** qui permet d'ajouter un produit au panier en prenant en considération si l'utilisateur est connecté ou non avec une vérification de la quantité et le stock du produit.

backend > cart > views.py > ...

```
1   from django.shortcuts import render,get_object_or_404,redirect
2
3   from django.views import View
4   from .models import CartItem,Cart
5   from products.models import Product
6
7
8   class AddToCartView(View):
9
10      def post(self,request,product_id):
11          product = get_object_or_404(Product, id=product_id)
12          try:
13              quantity = int(request.POST.get('quantity'))
14              if quantity<1:
15                  quantity=1
16          except ValueError:
17              quantity=1
18
19          if request.user.is_authenticated:
20              cart,_=Cart.objects.get_or_create(user=request.user)
21          else:
22              if not request.session.session_key:
23                  request.session.create()
24                  print(request.session.session_key)
25              cart,_=Cart.objects.get_or_create(session_key=request.session.session_key)
26          cart_item,created=CartItem.objects.get_or_create(cart=cart,product=product)
27          if created:
28              cart_item.quantity=quantity
29          else:
30              cart_item.quantity+=quantity
31          cart_item.save()
32          return redirect('cart_detail')
```

2) Associé la view AddToCartView à l'url suivant **(ligne 4)** :

urls.py

backend > cart > urls.py > ...

```
1  from django.urls import path
2  from .views import AddToCartView
3  urlpatterns = [
4      path('add/<int:product_id>',AddToCartView.as_view(),name='add_cart'),
5
6  ]
7
```

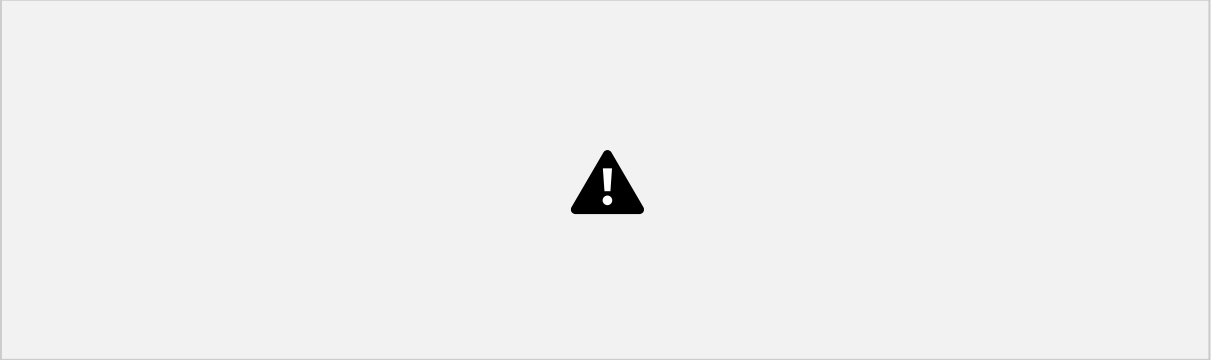
3) L'url associé à la vue AddToCartView doit être appelé lorsque l'utilisateur clique sur le bouton **Ajouter au panier** depuis la productDetail.html **(voir page4)**

2. Afficher les détails du panier

1) Créez une nouvelle vue appelée CartDetailsView dans le fichier views.py

− La méthode get_object() sert à **récupérer l'objet que la vue doit afficher**

2) Associé la view CartDetailView à l'url suivant (ligne 5) :



3) Créez une nouvelle template « detail_cart.html», au sein du dossiers templates de l'application «cart » pour afficher les détails du panier comme l'aperçu suivant :

− L'implémentation de la template details_cart.html est présentée à la page 8.

3. Supprimer un produit du panier

1) Créez une nouvelle vue appelée CartItemDeleteView dans le fichier views.py

2) Associé la vue à un url **(ligne 6)**

3) Créer une nouvelle template de confirmation de suppression « cartitem_delete.html »:

4) Editer la template « detail_cart.html» pour ajouter un lien qui permet de supprimer un produit du panier **(ligne 37)**

4. Modifier la quantité d'un produit du panier

1) Dans la template « detail_cart.html » chaque élément du panier (cartitem) affiche un formulaire permettant à l'utilisateur de mettre à jour la quantité du produit. Implémenter le code du formulaire

2) Ajouter une nouvelle vue **CartItemUpdateView** pour modifier la quantité l'élément du panier

3) Associer un url à la vue **CartItemUpdateView (ligne 7)**

4) Le formulaire définit précédemment dans la page 10 doit envoyer la quantité à la vue CartItemUpdateView

III. Création d'une page Inscription

1. Création d'un formulaire personnalisé

5) Pour permettez à Django de reconnaître et d'utiliser votre modèle personnalisé « CustomUser » pour toutes les opérations liées aux utilisateurs, telles que l'inscription, l'authentification et la gestion des sessions. Il faut modifier le fichier « settings.py » du projet backends et ajouter le paramètre suivant (AUTH_USER_MODEL) :

6) Créez un nouveau fichier forms.py dans l'application « users », pour personnaliser le formulaire d'inscription d'un nouvel utilisateur

7) Créez une nouvelle template « signup.html », au sein du dossiers templates de l'application «users »

2. Appliquer Bootstrap pour aux formulaires Django

Django Crispy Forms est une bibliothèque pour Django qui permet de personnaliser facilement les formulaires avec Bootstrap

1) Installer **Django Crispy Forms**

pip install django-crispy-forms

2) Installer **crispy-bootstrap5**

pip install crispy-bootstrap5

3) Dans le fichier settings.py:

− ajouter **'crispy_forms'** et **'crispy_bootstrap5' à** la liste des applications installées

− Configure Django pour utiliser Bootstrap 5

4) Editer le fichier « signup.html », afin d'utiliser bootstrap5 pour les rendu des formulaire

5) Dans le fichier views.py de l'application « users », créez une nouvelle vue appelée SignupView qui hérite de la classe générique CreateView de Django

6) Créez un fichier pour configurer les urls de l'application « users »

8) Inclure les urls de l'application users dans le projet Django «backend/urls.py »

9) Créez une template pour la connexion d’un utilisateur « users/login.htm »

10) Créez des vues pour la connexion et déconnexion d’un utilisateur

Remarque :

- **reverse_lazy('home'),** permet rediriger vers la page d'accueil

11) Éditez la vue d'inscription pour rediriger l'utilisateur après la création d'un compte vers la page de connexion à travers : **reverse_lazy('login')**

IV. Gestion du processus de commande

Travail à faire :

1) Créer dans l'application « orders » un fichier forms.py pour générer un formulaire depuis le modèle Order

2) Dans l'application cart créer un nouveau fichier appelé utils.py, qui permet de récupérer le panier du visiteur ou l'utilisateur connecté

3) Créez une nouvelle vue CreateOrderView et au sein de cette dernière créer la méthode get qui permet d'afficher le formulaire d'ajout d'une commande

4) Créez une url associée à cette vue

5) Créez la template shipping.html pour visualiser l’aperçu suivant

6) Ajouter dans la vue CreateOrderView, la méthode post qui permet de sauvegarder la commande

7) Créer la template confirmation.html pour afficher les details de la commande ajoutée